

Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Part 1:

You and your partner must share each other's Assignment 1 submission.

Part 2:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

```
In [ ]: # Question 1: check duplicates in tree
#The task is asking the user to analyse the tree structure and return nodes valu
#The difficulty of the task is that we need not only to return duplicates, but t
#If node is located closer to the root and there are multiple duplicates, we sho
```

- Create 1 new example that demonstrates you understand the problem.
Trace/walkthrough 1 example that your partner made and explain it.

```
In [22]: #Example from my side
#           1
#         2   2
#       3   3 4  4
root = TreeNode(1)
root.left = TreeNode(2)
root.right = TreeNode(2)
root.left.left = TreeNode(3)
root.left.right = TreeNode(3)
root.right.left = TreeNode(4)
root.right.right = TreeNode(4)
```

```
In [23]: is_duplicate(root)
# the code works correctly, the value of 2 is shown while we have other duplicat
```

Out[23]: 2

```
In [21]: #Example from partner
# Example 1:
# Tree shape:
#       1
#      / \
#     2   2
#    / \ / \
#   3  5 6  7
rootp = TreeNode(1)
rootp.left = TreeNode(2)
rootp.right = TreeNode(2)
rootp.left.left = TreeNode(3)
rootp.left.right = TreeNode(5)
rootp.right.left = TreeNode(6)
rootp.right.right = TreeNode(7)
is_duplicate(rootp)
```

Out[21]: 2

```
In [ ]: #Explanation:
#we start with empty set and node 1
#we assign to the node value of 1
#it is not in seen set (as seen set is blank)
#so we are on line

#seen.add(node.val)

#so now seen = {1}

#we append to the queue two values: 2 from left and 2 from right

#we start 2nd iteration with 2 from left:
#it is not in seen set (set = {1}), so we add it {1,2}
#we add to queue two nodes (3 and 5) from node 2 on the left

#we start 3rd iteration with 2 from right
# it is in seen set = {1,2}, so we return 2 and function stops

# we do not need to loop further as we found our duplicate
```

- Copy the solution your partner wrote.

```
In [ ]: class TreeNode(object):
    def __init__(self, val = 0, left = None, right = None):
        self.val = val
        self.left = left
        self.right = right

from collections import deque

def build_tree(levels):
    """
    Builds a binary tree from a level-order list.
    Example of a level-order list: [1, 10, 2, 3, 10, 12, 12]
    Returns the root node of the constructed binary tree.
    """
```

```

if not levels:
    return None

root = TreeNode(levels[0])
queue = deque([root])
i = 1 # Index to track position in 'levels'

# Use BFS to link children
while queue and i < len(levels):
    node = queue.popleft()

    # Left child
    if i < len(levels) and levels[i] is not None:
        node.left = TreeNode(levels[i])
        queue.append(node.left)
    i += 1

    # Right child
    if i < len(levels) and levels[i] is not None:
        node.right = TreeNode(levels[i])
        queue.append(node.right)
    i += 1

return root

def is_duplicate(root: TreeNode) -> int:
    # TODO

    if not root:
        return -1

    seen = set()
    queue = deque([root])

    while queue:
        node = queue.popleft()
        # Check if the node's value is already seen.
        if node.val in seen:
            return node.val
        seen.add(node.val)
        # Add children to the queue if they exist.
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)

    # No duplicate was found.
    return -1

```

- Explain why their solution works in your own words.

In []: *# The solution works because we start search in the tree from root > to leaf and
I believe it is a great solution as it stops as soon as duplicate was found
The first idea that came to my mind about this problem was to store node/dista
But it will not be the optimal solution*

- Explain the problem's time and space complexity in your own words.

```
In [ ]: # I think that time complexity is  $O(n)$  where  $n$  - number of nodes as we check eve
# Space complexity is also  $O(n)$  - size of set in the scenario where we have no d
```

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

```
In [ ]: # I have nothing to add here.
# I believe it is a great solution in terms of possible performance!
```

Part 3:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

```
In [ ]: In the assignment 1 I got question 2: storing tree paths in the separate list. T
In regard to my partner solution: I believe that he/she did a great work and fou
```

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution
- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

 **Please review our Assignment Submission Guide**  for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: HH:MM AM/PM - DD/MM/YYYY
- The branch name for your repo should be: assignment-2
- What to submit for this assignment:
 - This Jupyter Notebook (assignment_2.ipynb) should be populated and should be the only change in your pull request.

- What the pull request link should look like for this assignment:
`https://github.com/<your_github_username>/algorithms_and_data_structur`
 - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☐ Created a branch with the correct naming convention.
- ☐ Ensured that the repository is public.
- ☐ Reviewed the PR description guidelines and adhered to them.
- ☐ Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-3-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.